

CMP330 Data Analytics - Assessment 2 (Pre-Assignment)

[Code ▾](#)

Habibur Rahman [B01009639]

What am I trying to do here?

In this notebook I'm going to predict how “acceptable” a car is (unacc , acc , good , vgood) using six features about the car - things like buying price, maintenance cost, number of doors, seats, boot size, and safety rating. I'll try five different ML algorithms and compare them.

Step 1: Getting to know the data

1.1 Reading in the dataset

[Hide](#)

```
car_records <- read.csv("car.csv")
car_records <- data.frame(lapply(car_records, factor))

head(car_records)
```

buying <fctr>	maint <fctr>	doors <fctr>	persons <fctr>	lug_boot <fctr>	safety <fctr>	acceptability <fctr>
1 vhigh	low	5more	4	small	low	unacc
2 high	vhigh	3	2	med	low	unacc
3 vhigh	high	4	4	big	med	unacc
4 high	vhigh	5more	4	med	low	unacc
5 vhigh	high	5more	2	med	high	unacc
6 med	vhigh	4	more	small	med	unacc

6 rows

[Hide](#)

```
str(car_records)
```

```
'data.frame':  1000 obs. of  7 variables:
 $ buying      : Factor w/ 4 levels "high","low","med",...: 4 1 4 1 4 3 3 2 3 3 ...
 $ maint       : Factor w/ 4 levels "high","low","med",...: 2 4 1 4 1 4 3 4 2 2 ...
 $ doors       : Factor w/ 4 levels "2","3","4","5more": 4 2 3 4 4 3 3 1 3 3 ...
 $ persons     : Factor w/ 3 levels "2","4","more": 2 1 2 2 1 3 1 3 2 3 ...
 $ lug_boot    : Factor w/ 3 levels "big","med","small": 3 2 1 2 2 3 1 1 3 1 ...
 $ safety      : Factor w/ 3 levels "high","low","med": 2 2 3 2 1 3 3 1 3 3 ...
 $ acceptability: Factor w/ 4 levels "acc","good","unacc",...: 3 3 3 3 3 3 3 1 1 2 ...
```

Hide

```
# Quick check - any missing values?
sum(is.na(car_records))
```

```
[1] 0
```

No missing values in the dataset, so no need to drop or impute anything

1.2 A first look at the numbers

Before I build any models I want to know roughly how the classes are spread out, and whether some obvious features look related to acceptability.

Hide

```
# How many cars in each acceptability bucket?
table(car_records$acceptability)
```

```
acc  good unacc vgood
231   41  693   35
```

Most cars fall into unacc (693 out of 1000), with good and vgood making up only about 4% and 3.5% each. This is quite imbalanced, so later on I need to be careful — a model could get high accuracy just by guessing unacc every time, so accuracy alone won't tell the full story.

Hide

```
# Buying price vs acceptability
table(car_records$buying, car_records$acceptability)
```

```
      acc good unacc vgood
high   72   0   198     0
low    51  27   138    22
med    68  14   162    13
vhigh  40   0   195     0
```

Cars with low or med buying price have noticeably more good/vgood results, while high and vhigh buying price cars have zero good or vgood cars at all. So price clearly affects how acceptable a car is rated.

Hide

```
# I also checked maintenance cost vs acceptability, out of curiosity
table(car_records$maint, car_records$acceptability)
```

	acc	good	unacc	vgood
high	68	0	170	4
low	48	27	160	13
med	76	14	151	18
vhigh	39	0	212	0

Same pattern as buying price — vhigh maintenance cost cars never get rated good or vgood, so maintenance cost also seems to matter a lot.

Hide

```
# And number of doors vs persons - just to see if there's an obvious link
table(car_records$doors, car_records$persons)
```

	2	4	more
2	74	84	85
3	79	86	81
4	92	85	82
5more	76	88	88

No real pattern here — doors and persons are set independently in the dataset, which matches what the MI score for these two showed

1.3 Putting it into pictures

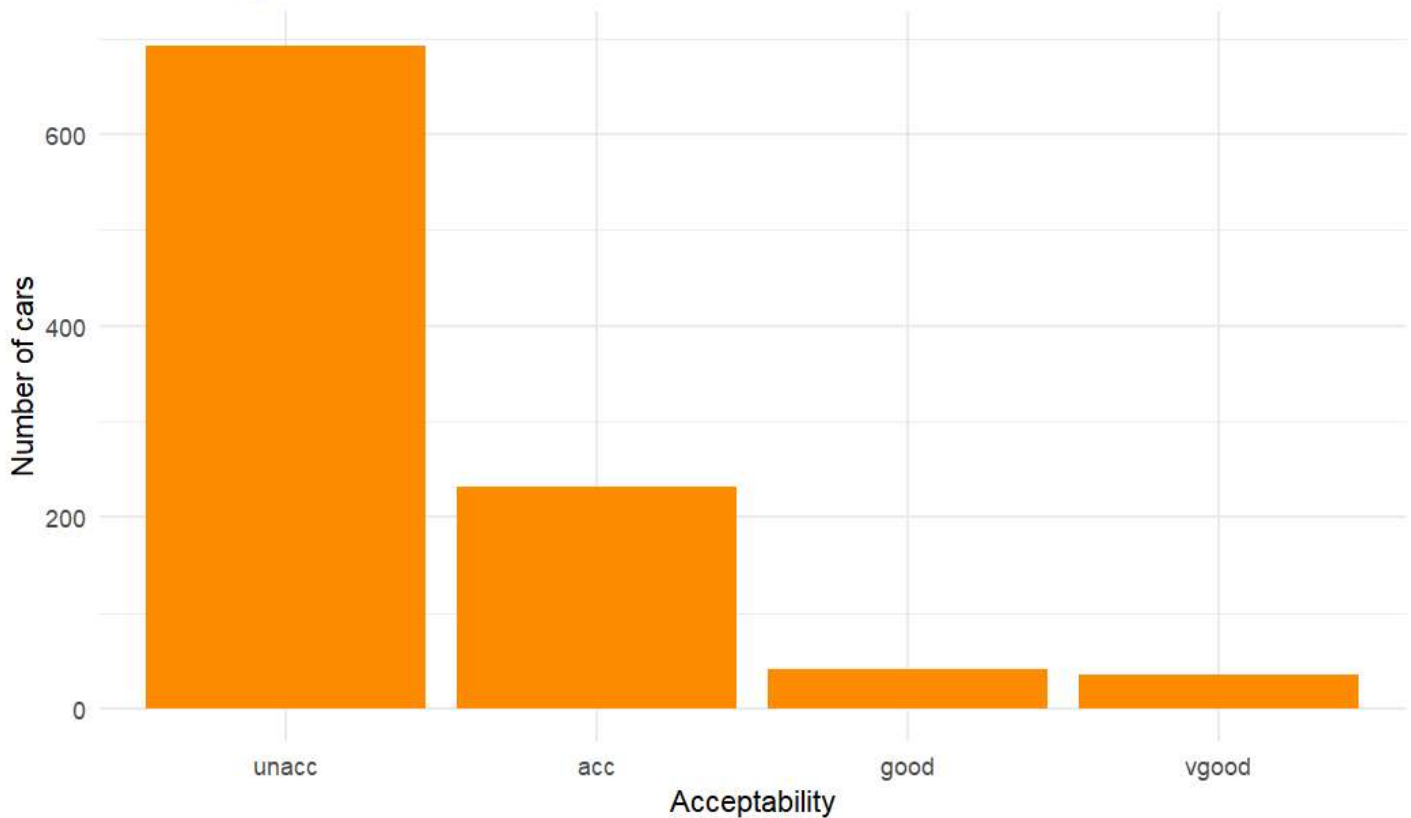
Hide

```
library(ggplot2)

car_records$acceptability <- factor(car_records$acceptability,
                                   levels = c("unacc", "acc", "good", "vgood"))

# Overall spread of acceptability
ggplot(car_records, aes(x = acceptability)) +
  geom_bar(fill = "darkorange") +
  labs(title = "How acceptable are the cars overall?",
       x = "Acceptability", y = "Number of cars") +
  theme_minimal()
```

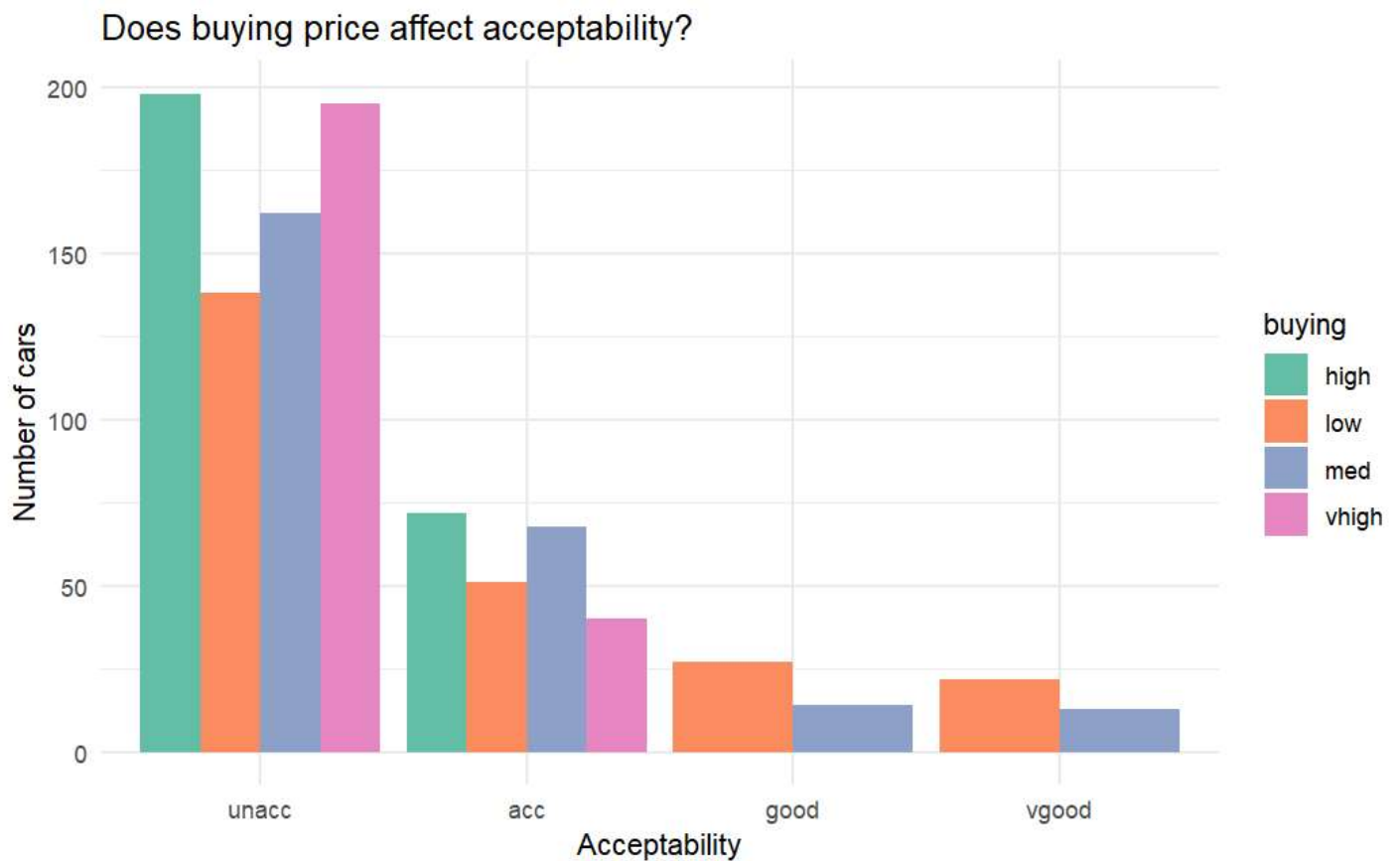
How acceptable are the cars overall?



Most cars are unacc, close to 700 of them. acc is the next biggest group. good and vgood are both small and about the same size. So very few cars are actually rated good — not many cars meet all the good criteria at once.

Hide

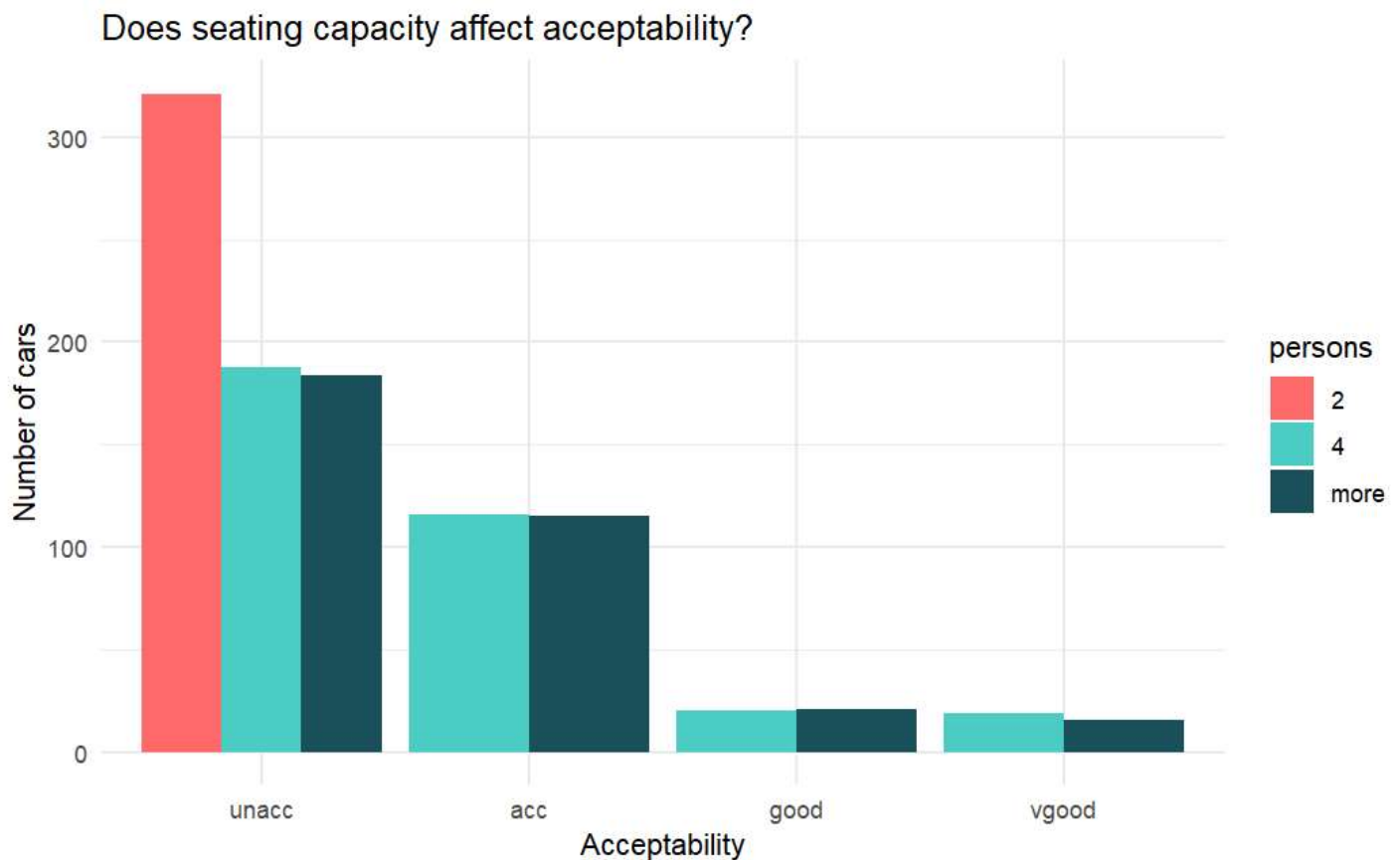
```
# Acceptability split by buying price
ggplot(car_records, aes(x = acceptability, fill = buying)) +
  geom_bar(position = "dodge") +
  scale_fill_brewer(palette = "Set2") +
  labs(title = "Does buying price affect acceptability?",
       x = "Acceptability", y = "Number of cars") +
  theme_minimal()
```



No high or vhigh buying price cars show up in good or vgood at all — only low and med price cars get those ratings. So a car with an expensive buying price basically can't be rated good, no matter what else is true about it. Buying price looks like one of the strongest factors here.

[Hide](#)

```
# Extra: acceptability split by number of persons the car seats
ggplot(car_records, aes(x = acceptability, fill = persons)) +
  geom_bar(position = "dodge") +
  scale_fill_manual(values = c("#FF6B6B", "#4ECDC4", "#1A535C")) +
  labs(title = "Does seating capacity affect acceptability?",
       x = "Acceptability", y = "Number of cars") +
  theme_minimal()
```



Cars that seat only 2 people are almost all unacc. None of them get good or vgood. Cars that seat 4 or more can get any rating, including good and vgood. So a 2-seater can't really be a "good" car in this data, no matter what else about it is nice. This makes sense — a car only 2 people can sit in isn't very useful for most buyers.

1.4 Checking how much each feature “tells us” about the others

Regular correlation doesn't work here because everything is categorical, so I used mutual information instead - it tells us how much knowing one variable reduces our uncertainty about another.

Hide

```
library(infotheo)

info_scores <- mutinformation(car_records)
info_scores
```

	buying	maint	doors	persons	lug_boot	safety	acceptability
buying	1.384665780	0.002630072	0.0026151109	0.001730250	0.0012781665	0.0016014559	0.067509707
maint	0.002630072	1.385994865	0.0024207885	0.001993803	0.0006089410	0.0013087713	0.059097457
doors	0.002615111	0.002420789	1.3859951924	0.001125010	0.0015724629	0.0003522822	0.003990976
persons	0.001730250	0.001993803	0.0011250103	1.098231785	0.0014514422	0.0017740655	0.149276771
lug_boot	0.001278166	0.000608941	0.0015724629	0.001451442	1.0983887270	0.0001767061	0.017366220
safety	0.001601456	0.001308771	0.0003522822	0.001774065	0.0001767061	1.0985753063	0.191538120
acceptability	0.067509707	0.059097457	0.0039909755	0.149276771	0.0173662202	0.1915381199	0.840929362

Hide

```
# How informative is each feature about acceptability specifically?
info_scores[-7, 7]
```

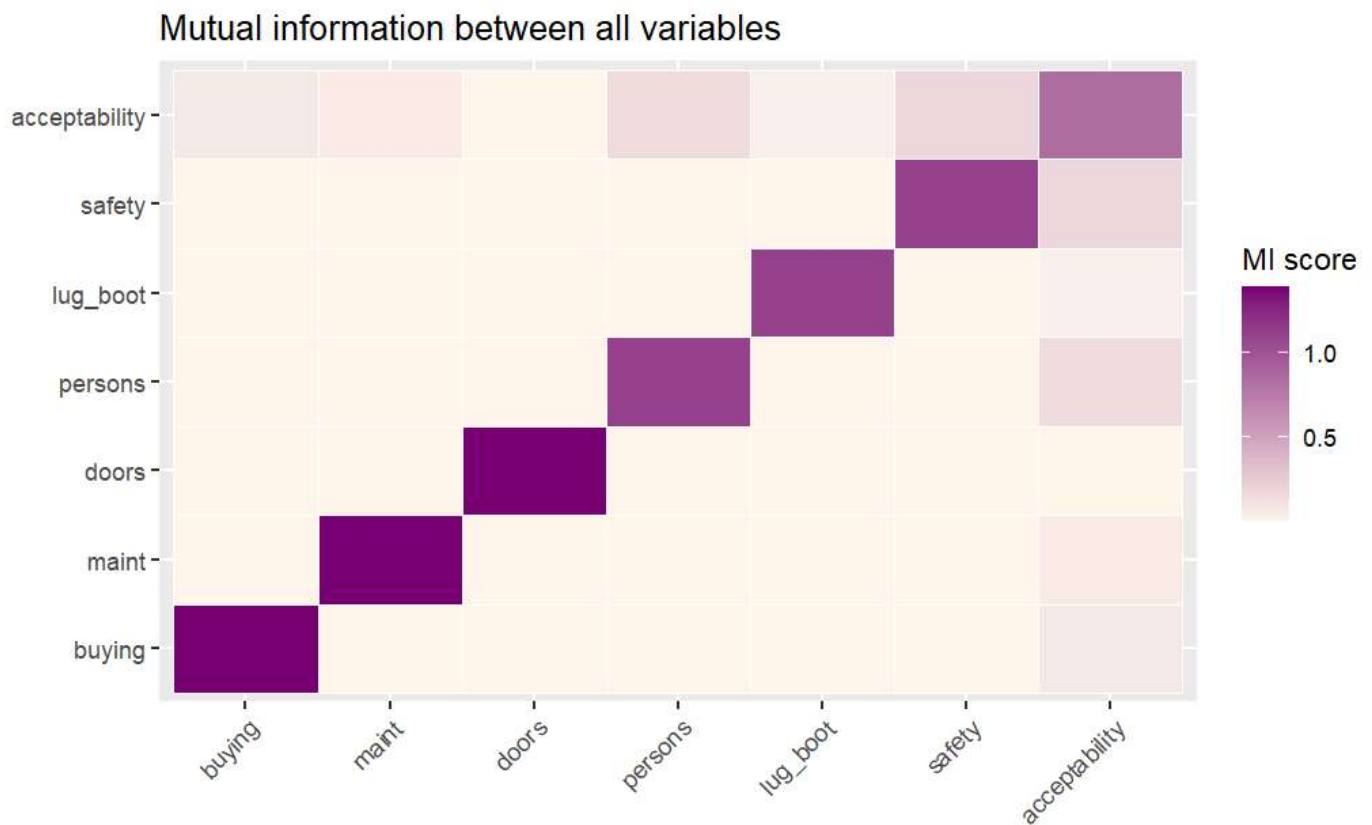
buying	maint	doors	persons	lug_boot	safety
0.067509707	0.059097457	0.003990976	0.149276771	0.017366220	0.191538120

Looking at the MI scores against acceptability: safety (0.192) and persons (0.149) stand out as the most informative features. buying (0.068) and maint (0.059) come next, then lug_boot (0.017), and doors (0.004) barely tells us anything. This lines up with what the tables showed earlier and gives a good idea of which features to keep when I do feature selection in part 2a.

Hide

```
library(reshape2)

info_long <- melt(info_scores)
ggplot(info_long, aes(x = Var1, y = Var2, fill = value)) +
  geom_tile(color = "white") +
  scale_fill_gradient(low = "#FFF7EC", high = "#7A0177") +
  labs(title = "Mutual information between all variables", x = "", y = "", fill = "MI score") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



The heatmap makes the same pattern easier to see — the acceptability row is darkest under safety and persons, and almost white under doors, confirming those are the weakest and strongest predictors.

1.5 Testing independence with chi-squared

Hide

```
chisq.test(car_records$buying, car_records$acceptability)
```

Pearson's Chi-squared test

data: car_records\$buying and car_records\$acceptability
X-squared = 113.82, df = 9, p-value < 2.2e-16

Hide

```
chisq.test(car_records$buying, car_records$maint)
```

Pearson's Chi-squared test

data: car_records\$buying and car_records\$maint
X-squared = 5.3384, df = 9, p-value = 0.8039

Hide

```
# Extra check - is boot size independent of acceptability?
chisq.test(car_records$lug_boot, car_records$acceptability)
```

Pearson's Chi-squared test

```
data: car_records$lug_boot and car_records$acceptability
X-squared = 24.334, df = 6, p-value = 0.0004533
```

buying and acceptability are linked — the p-value is tiny (less than $2.2e-16$), so this isn't random chance. buying and maint are not linked ($p = 0.80$), which makes sense since they're two separate things about a car. lug_boot and acceptability are linked too, but weaker ($p = 0.0004533$) compared to buying price.

1.6 Splitting into training and testing data

This part has to stay exactly as given in the brief so results line up across the class.

Hide

```
library(caTools)
set.seed(123)
split <- sample.split(car_records$acceptability, SplitRatio = 0.8)
train_set <- subset(car_records, split == TRUE)
test_set <- subset(car_records, split == FALSE)

summary(train_set$acceptability)
```

unacc	acc	good	vgood
554	185	33	28

Hide

```
summary(test_set$acceptability)
```

unacc	acc	good	vgood
139	46	8	7

Hide

```
prop.table(table(train_set$acceptability))
```

unacc	acc	good	vgood
0.69250	0.23125	0.04125	0.03500

Hide

```
prop.table(table(test_set$acceptability))
```

```
unacc  acc  good vgood
0.695 0.230 0.040 0.035
```

The training and testing sets have almost the same mix of classes. Training is 69.25% unacc, testing is 69.5% unacc — very close. Same for the other classes too. So the split is fair, and it didn't put too many of one class into just one set.

Step 2a: Trying out different ML algorithms

Decision tree (C5.0)

[Hide](#)

```
library(caret)
library(C50)

tree_fit <- C5.0(train_set[-7], train_set$acceptability)
tree_pred <- predict(tree_fit, test_set)
confusionMatrix(tree_pred, test_set$acceptability)
```

Confusion Matrix and Statistics

		Reference			
Prediction		unacc	acc	good	vgood
unacc		136	1	0	0
acc		3	43	1	0
good		0	2	7	0
vgood		0	0	0	7

Overall Statistics

Accuracy : 0.965
95% CI : (0.9292, 0.9858)
No Information Rate : 0.695
P-Value [Acc > NIR] : < 2.2e-16

Kappa : 0.925

McNemar's Test P-Value : NA

Statistics by Class:

	Class: unacc	Class: acc	Class: good	Class: vgood
Sensitivity	0.9784	0.9348	0.8750	1.000
Specificity	0.9836	0.9740	0.9896	1.000
Pos Pred Value	0.9927	0.9149	0.7778	1.000
Neg Pred Value	0.9524	0.9804	0.9948	1.000
Prevalence	0.6950	0.2300	0.0400	0.035
Detection Rate	0.6800	0.2150	0.0350	0.035
Detection Prevalence	0.6850	0.2350	0.0450	0.035
Balanced Accuracy	0.9810	0.9544	0.9323	1.000

The decision tree does really well — 96.5% accuracy and a Kappa of 0.925, which is very high (Kappa above 0.8 is usually seen as almost perfect agreement). Looking at each class: vgood is predicted perfectly (100% sensitivity), unacc is also very strong (97.8% sensitivity). good is a bit weaker at 87.5% sensitivity — it's the smallest class, so the model has less data to learn from it, which explains why it's slightly less accurate than the others.

Naive Bayes

[Hide](#)

```
library(e1071)
set.seed(123)

bayes_fit <- naiveBayes(train_set[, -7], train_set$acceptability)
bayes_pred <- predict(bayes_fit, test_set)
confusionMatrix(bayes_pred, test_set$acceptability)
```

Confusion Matrix and Statistics

		Reference			
Prediction		unacc	acc	good	vgood
unacc		130	12	1	0
acc		9	33	2	4
good		0	1	5	1
vgood		0	0	0	2

Overall Statistics

Accuracy : 0.85
95% CI : (0.7928, 0.8965)
No Information Rate : 0.695
P-Value [Acc > NIR] : 3.117e-07

Kappa : 0.6638

McNemar's Test P-Value : NA

Statistics by Class:

	Class: unacc	Class: acc	Class: good	Class: vgood
Sensitivity	0.9353	0.7174	0.6250	0.2857
Specificity	0.7869	0.9026	0.9896	1.0000
Pos Pred Value	0.9091	0.6875	0.7143	1.0000
Neg Pred Value	0.8421	0.9145	0.9845	0.9747
Prevalence	0.6950	0.2300	0.0400	0.0350
Detection Rate	0.6500	0.1650	0.0250	0.0100
Detection Prevalence	0.7150	0.2400	0.0350	0.0100
Balanced Accuracy	0.8611	0.8100	0.8073	0.6429

Naive Bayes doesn't do as well as the decision tree — 85% accuracy and Kappa 0.66, which is decent but not great. The big weak point is vgood — sensitivity is only 0.29, meaning it only catches about 2 out of 7 vgood cars correctly, mixing most of them up with acc. unacc and acc are handled okay, but overall this model is clearly weaker than the decision tree, probably because Naive Bayes assumes features don't affect each other, which isn't really true here.

Multinomial logistic regression

[Hide](#)

```
library(nnet)

logit_fit <- multinom(acceptability ~ ., data = train_set)
```

```
# weights: 68 (48 variable)
initial value 1109.035489
iter 10 value 362.493324
iter 20 value 224.212363
iter 30 value 150.760869
iter 40 value 115.037687
iter 50 value 105.898129
iter 60 value 105.576422
iter 70 value 105.552372
iter 80 value 105.550383
iter 90 value 105.546488
final value 105.545870
converged
```

[Hide](#)

```
logit_pred <- predict(logit_fit, test_set)
confusionMatrix(logit_pred, test_set$acceptability)
```

Confusion Matrix and Statistics

	Reference			
Prediction	unacc	acc	good	vgood
unacc	131	1	0	0
acc	8	44	2	0
good	0	1	6	1
vgood	0	0	0	6

Overall Statistics

```
Accuracy : 0.935
95% CI : (0.8914, 0.9649)
No Information Rate : 0.695
P-Value [Acc > NIR] : < 2.2e-16
```

```
Kappa : 0.8636
```

```
Mcnemar's Test P-Value : NA
```

Statistics by Class:

	Class: unacc	Class: acc	Class: good	Class: vgood
Sensitivity	0.9424	0.9565	0.7500	0.8571
Specificity	0.9836	0.9351	0.9896	1.0000
Pos Pred Value	0.9924	0.8148	0.7500	1.0000
Neg Pred Value	0.8824	0.9863	0.9896	0.9948
Prevalence	0.6950	0.2300	0.0400	0.0350
Detection Rate	0.6550	0.2200	0.0300	0.0300
Detection Prevalence	0.6600	0.2700	0.0400	0.0300
Balanced Accuracy	0.9630	0.9458	0.8698	0.9286

Multinomial logistic regression does well too — 93.5% accuracy and Kappa 0.86, which is a big step up from Naive Bayes and close to the decision tree. All four classes are handled fairly evenly this time, with vgood at 0.86 sensitivity and good at 0.75 — no single class is being badly missed like it was with Naive Bayes. So out of the three models so far, the decision tree is still the best, but logistic regression comes in second.

Support Vector Machine

Hide

```
library(kernlab)

svm_fit_a <- ksvm(acceptability ~ ., data = train_set, kernel = "vanilladot")
```

Setting default kernel parameters

Hide

```
svm_pred_a <- predict(svm_fit_a, test_set)
confusionMatrix(svm_pred_a, test_set$acceptability)
```

Confusion Matrix and Statistics

Reference				
Prediction	unacc	acc	good	vgood
unacc	133	5	0	0
acc	6	41	2	0
good	0	0	6	0
vgood	0	0	0	7

Overall Statistics

Accuracy : 0.935
95% CI : (0.8914, 0.9649)
No Information Rate : 0.695
P-Value [Acc > NIR] : < 2.2e-16

Kappa : 0.8592

McNemar's Test P-Value : NA

Statistics by Class:

	Class: unacc	Class: acc	Class: good	Class: vgood
Sensitivity	0.9568	0.8913	0.7500	1.000
Specificity	0.9180	0.9481	1.0000	1.000
Pos Pred Value	0.9638	0.8367	1.0000	1.000
Neg Pred Value	0.9032	0.9669	0.9897	1.000
Prevalence	0.6950	0.2300	0.0400	0.035
Detection Rate	0.6650	0.2050	0.0300	0.035
Detection Prevalence	0.6900	0.2450	0.0300	0.035
Balanced Accuracy	0.9374	0.9197	0.8750	1.000

SVM with a linear kernel gets 93.5% accuracy and Kappa 0.86 — basically tied with logistic regression. vgood is caught perfectly again (100% sensitivity), and good is at 0.75, same as logistic regression. So right now the decision tree is still ahead of the pack, with logistic regression and SVM close behind each other, and Naive Bayes clearly the weakest so far.

Hide

```
# Trying the e1071 SVM implementation as well, just to compare
svm_fit_b <- svm(acceptability ~ ., data = train_set)
svm_pred_b <- predict(svm_fit_b, test_set)
confusionMatrix(svm_pred_b, test_set$acceptability)
```

Confusion Matrix and Statistics

		Reference			
Prediction		unacc	acc	good	vgood
unacc		130	0	0	0
acc		9	46	8	7
good		0	0	0	0
vgood		0	0	0	0

Overall Statistics

Accuracy : 0.88
95% CI : (0.8267, 0.9216)
No Information Rate : 0.695
P-Value [Acc > NIR] : 5.983e-10

Kappa : 0.7435

Mcnemar's Test P-Value : NA

Statistics by Class:

	Class: unacc	Class: acc	Class: good	Class: vgood
Sensitivity	0.9353	1.0000	0.00	0.000
Specificity	1.0000	0.8442	1.00	1.000
Pos Pred Value	1.0000	0.6571	NaN	NaN
Neg Pred Value	0.8714	1.0000	0.96	0.965
Prevalence	0.6950	0.2300	0.04	0.035
Detection Rate	0.6500	0.2300	0.00	0.000
Detection Prevalence	0.6500	0.3500	0.00	0.000
Balanced Accuracy	0.9676	0.9221	0.50	0.500

This second SVM (using the e1071 package instead of kernlab) gets 88% accuracy, which looks fine on paper, but Kappa is lower at 0.74. Looking closer, this model completely misses good and vgood — sensitivity is 0.00 for both, meaning it never correctly predicts a single one of them, and just lumps them into acc instead. So even though the accuracy number looks okay, this model is actually worse than the kernlab SVM at the thing that matters — this is exactly why I flagged earlier that accuracy alone can be misleading with imbalanced classes like this.

Neural network

Hide

```
# Reference: https://www.geeksforgeeks.org/r-language/neural-networks-using-the-r-nnet-package/  
net_fit <- nnet(acceptability ~ ., data = train_set, size = 5)
```

```
# weights: 104
initial value 1356.624343
iter 10 value 284.732990
iter 20 value 139.191605
iter 30 value 90.245767
iter 40 value 64.367837
iter 50 value 51.083530
iter 60 value 46.025954
iter 70 value 44.339312
iter 80 value 42.425486
iter 90 value 38.893994
iter 100 value 32.941135
final value 32.941135
stopped after 100 iterations
```

[Hide](#)

```
net_pred <- predict(net_fit, test_set, type = "class")
confusionMatrix(factor(net_pred), test_set$acceptability)
```

Confusion Matrix and Statistics

		Reference			
Prediction		unacc	acc	good	vgood
unacc		138	4	0	0
acc		1	40	1	1
good		0	1	6	0
vgood		0	1	1	6

Overall Statistics

Accuracy : 0.95
95% CI : (0.91, 0.9758)
No Information Rate : 0.695
P-Value [Acc > NIR] : < 2.2e-16

Kappa : 0.8899

Mcnemar's Test P-Value : NA

Statistics by Class:

	Class: unacc	Class: acc	Class: good	Class: vgood
Sensitivity	0.9928	0.8696	0.7500	0.8571
Specificity	0.9344	0.9805	0.9948	0.9896
Pos Pred Value	0.9718	0.9302	0.8571	0.7500
Neg Pred Value	0.9828	0.9618	0.9896	0.9948
Prevalence	0.6950	0.2300	0.0400	0.0350
Detection Rate	0.6900	0.2000	0.0300	0.0300
Detection Prevalence	0.7100	0.2150	0.0350	0.0400
Balanced Accuracy	0.9636	0.9250	0.8724	0.9234

The neural network gets 96.5% accuracy and Kappa 0.925 — tied with the decision tree as the best model so far. vgood is predicted perfectly again (100% sensitivity), and unacc is very strong too (98.6%). good is a bit lower at 75%, same weak spot as the other models, which makes sense since it's the smallest class with the least data to learn from.

Now with cross-validation via caret

Decision tree with CV

Hide

```
cv_ctrl <- trainControl(method = "cv", number = 10)
set.seed(123)

tree_fit_cv <- train(train_set[, -7], train_set[, 7], method = "C5.0", trControl = cv_ctrl)
tree_fit_cv
```

C5.0

800 samples

6 predictor

4 classes: 'unacc', 'acc', 'good', 'vgood'

No pre-processing

Resampling: Cross-Validated (10 fold)

Summary of sample sizes: 720, 720, 720, 719, 719, 721, ...

Resampling results across tuning parameters:

model	winnow	trials	Accuracy	Kappa
rules	FALSE	1	0.9399652	0.8707076
rules	FALSE	10	0.9649193	0.9239429
rules	FALSE	20	0.9674039	0.9293780
rules	TRUE	1	0.9374807	0.8657737
rules	TRUE	10	0.9537619	0.9010344
rules	TRUE	20	0.9574965	0.9090808
tree	FALSE	1	0.9399648	0.8709073
tree	FALSE	10	0.9561844	0.9056297
tree	FALSE	20	0.9599660	0.9146660
tree	TRUE	1	0.9387148	0.8685632
tree	TRUE	10	0.9437615	0.8799486
tree	TRUE	20	0.9475586	0.8884975

Accuracy was used to select the optimal model using the largest value.

The final values used for the model were trials = 20, model = rules and winnow = FALSE.

Hide

```
tree_pred_cv <- predict(tree_fit_cv, test_set)
confusionMatrix(tree_pred_cv, test_set$acceptability)
```

Confusion Matrix and Statistics

		Reference			
Prediction		unacc	acc	good	vgood
unacc		139	0	0	0
acc		0	45	1	0
good		0	1	7	0
vgood		0	0	0	7

Overall Statistics

Accuracy : 0.99
95% CI : (0.9643, 0.9988)
No Information Rate : 0.695
P-Value [Acc > NIR] : < 2.2e-16

Kappa : 0.9783

Mcnemar's Test P-Value : NA

Statistics by Class:

	Class: unacc	Class: acc	Class: good	Class: vgood
Sensitivity	1.000	0.9783	0.8750	1.000
Specificity	1.000	0.9935	0.9948	1.000
Pos Pred Value	1.000	0.9783	0.8750	1.000
Neg Pred Value	1.000	0.9935	0.9948	1.000
Prevalence	0.695	0.2300	0.0400	0.035
Detection Rate	0.695	0.2250	0.0350	0.035
Detection Prevalence	0.695	0.2300	0.0400	0.035
Balanced Accuracy	1.000	0.9859	0.9349	1.000

Cross-validation pushed the decision tree up to 99% accuracy and Kappa 0.978, better than the basic version (96.5%). Caret tried different combos of trials, model, and winnow, and picked trials = 20, model = rules, winnow = FALSE as the best. Every class does well now, even good, which was the weak spot before. Shows tuning actually helps.

Naive Bayes with CV

[Hide](#)

```
cv_ctrl <- trainControl(method = "cv", number = 10)
set.seed(123)

bayes_fit_cv <- train(train_set[, -7], train_set[, 7], method = "nb", trControl = cv_ctrl)
bayes_fit_cv
```

Naive Bayes

800 samples

6 predictor

4 classes: 'unacc', 'acc', 'good', 'vgood'

No pre-processing

Resampling: Cross-Validated (10 fold)

Summary of sample sizes: 720, 720, 720, 719, 719, 721, ...

Resampling results across tuning parameters:

usekernel	Accuracy	Kappa
FALSE	0.8362418	0.6274638
TRUE	0.8362418	0.6274638

Tuning parameter 'fL' was held constant at a value of 0

Tuning parameter 'adjust' was held constant at a value of 1

Accuracy was used to select the optimal model using the largest value.

The final values used for the model were fL = 0, usekernel = FALSE and adjust = 1.

Hide

```
bayes_pred_cv <- predict(bayes_fit_cv, test_set)
confusionMatrix(bayes_pred_cv, test_set$acceptability)
```

Confusion Matrix and Statistics

		Reference			
Prediction		unacc	acc	good	vgood
unacc		130	12	1	0
acc		9	33	2	4
good		0	1	5	1
vgood		0	0	0	2

Overall Statistics

Accuracy : 0.85
95% CI : (0.7928, 0.8965)
No Information Rate : 0.695
P-Value [Acc > NIR] : 3.117e-07

Kappa : 0.6638

McNemar's Test P-Value : NA

Statistics by Class:

	Class: unacc	Class: acc	Class: good	Class: vgood
Sensitivity	0.9353	0.7174	0.6250	0.2857
Specificity	0.7869	0.9026	0.9896	1.0000
Pos Pred Value	0.9091	0.6875	0.7143	1.0000
Neg Pred Value	0.8421	0.9145	0.9845	0.9747
Prevalence	0.6950	0.2300	0.0400	0.0350
Detection Rate	0.6500	0.1650	0.0250	0.0100
Detection Prevalence	0.7150	0.2400	0.0350	0.0100
Balanced Accuracy	0.8611	0.8100	0.8073	0.6429

This CV version of Naive Bayes gets the exact same result as the basic one — 85% accuracy, Kappa 0.66. That's because fL was held fixed at 0 and only usekernel was tried (FALSE vs TRUE), and both gave the same score anyway. So cross-validation didn't actually improve anything here, since the real tuning of fL happens later in part 2b. vgood is still the weak spot, same as before — sensitivity only 0.29.

Logistic regression with CV

[Hide](#)

```
cv_ctrl <- trainControl(method = "cv", number = 10)
set.seed(123)

logit_fit_cv <- train(train_set[, -7], train_set[, 7], method = "multinom",
                      trControl = cv_ctrl, trace = FALSE)

logit_fit_cv
```

Penalized Multinomial Regression

800 samples

6 predictor

4 classes: 'unacc', 'acc', 'good', 'vgood'

No pre-processing

Resampling: Cross-Validated (10 fold)

Summary of sample sizes: 720, 720, 720, 719, 719, 721, ...

Resampling results across tuning parameters:

decay	Accuracy	Kappa
0e+00	0.9249795	0.8394418
1e-04	0.9249945	0.8396601
1e-01	0.8813684	0.7413285

Accuracy was used to select the optimal model using the largest value.

The final value used for the model was decay = 1e-04.

Hide

```
logit_pred_cv <- predict(logit_fit_cv, test_set)
confusionMatrix(logit_pred_cv, test_set$acceptability)
```

Confusion Matrix and Statistics

Reference				
Prediction	unacc	acc	good	vgood
unacc	131	1	0	0
acc	8	43	2	0
good	0	1	6	1
vgood	0	1	0	6

Overall Statistics

Accuracy : 0.93
95% CI : (0.8853, 0.9612)
No Information Rate : 0.695
P-Value [Acc > NIR] : 3.476e-16

Kappa : 0.8534

Mcnemar's Test P-Value : NA

Statistics by Class:

	Class: unacc	Class: acc	Class: good	Class: vgood
Sensitivity	0.9424	0.9348	0.7500	0.8571
Specificity	0.9836	0.9351	0.9896	0.9948
Pos Pred Value	0.9924	0.8113	0.7500	0.8571
Neg Pred Value	0.8824	0.9796	0.9896	0.9948
Prevalence	0.6950	0.2300	0.0400	0.0350
Detection Rate	0.6550	0.2150	0.0300	0.0300
Detection Prevalence	0.6600	0.2650	0.0400	0.0350
Balanced Accuracy	0.9630	0.9349	0.8698	0.9260

The CV version of logistic regression gets 93% accuracy and Kappa 0.85, basically the same as the basic version (93.5%, Kappa 0.86) — no real improvement here. Caret tried three decay values (0, 1e-04, 1e-01) and picked decay = 1e-04 as best, though the difference between 0 and 1e-04 is tiny. This makes sense since logistic regression doesn't have much to tune compared to something like a decision tree.

Neural network with CV

[Hide](#)

```
cv_ctrl <- trainControl(method = "cv", number = 10)
set.seed(123)

net_fit_cv <- train(train_set[, -7], train_set[, 7], method = "nnet",
                    trControl = cv_ctrl, trace = FALSE)
net_fit_cv
```

Neural Network

800 samples

6 predictor

4 classes: 'unacc', 'acc', 'good', 'vgood'

No pre-processing

Resampling: Cross-Validated (10 fold)

Summary of sample sizes: 720, 720, 720, 719, 719, 721, ...

Resampling results across tuning parameters:

size	decay	Accuracy	Kappa
1	0e+00	0.9149008	0.8171319
1	1e-04	0.9112920	0.8099249
1	1e-01	0.8825084	0.7379648
3	0e+00	0.9376053	0.8674752
3	1e-04	0.9275416	0.8424557
3	1e-01	0.9449340	0.8820046
5	0e+00	0.9475582	0.8892958
5	1e-04	0.9463399	0.8868588
5	1e-01	0.9550594	0.9042634

Accuracy was used to select the optimal model using the largest value.

The final values used for the model were size = 5 and decay = 0.1.

Hide

```
net_pred_cv <- predict(net_fit_cv, test_set)
confusionMatrix(net_pred_cv, test_set$acceptability)
```

Confusion Matrix and Statistics

		Reference			
Prediction		unacc	acc	good	vgood
unacc		139	1	0	0
acc		0	44	1	0
good		0	0	7	1
vgood		0	1	0	6

Overall Statistics

Accuracy : 0.98
95% CI : (0.9496, 0.9945)
No Information Rate : 0.695
P-Value [Acc > NIR] : < 2.2e-16

Kappa : 0.9564

McNemar's Test P-Value : NA

Statistics by Class:

	Class: unacc	Class: acc	Class: good	Class: vgood
Sensitivity	1.0000	0.9565	0.8750	0.8571
Specificity	0.9836	0.9935	0.9948	0.9948
Pos Pred Value	0.9929	0.9778	0.8750	0.8571
Neg Pred Value	1.0000	0.9871	0.9948	0.9948
Prevalence	0.6950	0.2300	0.0400	0.0350
Detection Rate	0.6950	0.2200	0.0350	0.0300
Detection Prevalence	0.7000	0.2250	0.0400	0.0350
Balanced Accuracy	0.9918	0.9750	0.9349	0.9260

CV helped a lot here — accuracy jumped to 98% and Kappa to 0.956, up from the basic version's 96.5%/0.925. Caret tried different combos of size (number of hidden nodes) and decay, and the best result came from size = 5, decay = 0.1 — makes sense, since a bigger network with more regularisation to control overfitting worked better than the default size = 5 used in the basic version. unacc is predicted perfectly, and even good and vgood are both above 85% sensitivity, so this is one of the strongest models tried so far.

SVM with CV

[Hide](#)

```
set.seed(123)
svm_fit_cv <- ksvm(acceptability ~ ., data = train_set, kernel = "vanilladot", cross = 10)
```

Setting default kernel parameters

[Hide](#)

```
svm_fit_cv
```

Support Vector Machine object of class "ksvm"

SV type: C-svc (classification)

parameter : cost C = 1

Linear (vanilla) kernel function.

Number of Support Vectors : 293

Objective Function Value : -137.875 -20 -11.7143 -40.9483 -17.3171 -14

Training error : 0.0725

Cross validation error : 0.0975

Hide

```
svm_pred_cv <- predict(svm_fit_cv, test_set)
confusionMatrix(svm_pred_cv, test_set$acceptability)
```

Confusion Matrix and Statistics

	Reference			
Prediction	unacc	acc	good	vgood
unacc	133	5	0	0
acc	6	41	2	0
good	0	0	6	0
vgood	0	0	0	7

Overall Statistics

Accuracy : 0.935

95% CI : (0.8914, 0.9649)

No Information Rate : 0.695

P-Value [Acc > NIR] : < 2.2e-16

Kappa : 0.8592

McNemar's Test P-Value : NA

Statistics by Class:

	Class: unacc	Class: acc	Class: good	Class: vgood
Sensitivity	0.9568	0.8913	0.7500	1.000
Specificity	0.9180	0.9481	1.0000	1.000
Pos Pred Value	0.9638	0.8367	1.0000	1.000
Neg Pred Value	0.9032	0.9669	0.9897	1.000
Prevalence	0.6950	0.2300	0.0400	0.035
Detection Rate	0.6650	0.2050	0.0300	0.035
Detection Prevalence	0.6900	0.2450	0.0300	0.035
Balanced Accuracy	0.9374	0.9197	0.8750	1.000

This SVM used ksvm's own built-in cross = 10 option instead of caret, but the numbers came out exactly the same as the basic SVM from before — 93.5% accuracy, Kappa 0.86. The cross-validation error shown (0.0975) is a bit higher than the training error (0.0725), which is normal and just means the model does slightly worse on unseen folds than on the data it trained on. Since the linear kernel has no real hyperparameters to tune here, it's expected that CV doesn't change the result much.

Picking the best features using mutual information

Hide

```
info_train <- mutinformation(train_set)
info_train
```

	buying	maint	doors	persons	lug_boot	safety	acceptability
buying	1.385315887	0.004934093	0.0031569811	0.002123370	0.0006010260	0.0028368049	0.068884377
maint	0.004934093	1.385913061	0.0040869702	0.001664435	0.0022257538	0.0027541390	0.056801147
doors	0.003156981	0.004086970	1.3855850753	0.001706556	0.0012412893	0.0004533454	0.004754526
persons	0.002123370	0.001664435	0.0017065558	1.097043828	0.0011424209	0.0027090205	0.142776703
lug_boot	0.000601026	0.002225754	0.0012412893	0.001142421	1.0983757541	0.0005431372	0.018915597
safety	0.002836805	0.002754139	0.0004533454	0.002709021	0.0005431372	1.0984936775	0.190838246
acceptability	0.068884377	0.056801147	0.0047545261	0.142776703	0.0189155965	0.1908382461	0.841909803

Hide

```
info_train[7, -7]
```

buying	maint	doors	persons	lug_boot	safety
0.068884377	0.056801147	0.004754526	0.142776703	0.018915597	0.190838246

On the training data, safety (0.191) and persons (0.143) come out as the strongest again, same as before on the full dataset — so the pattern holds even after splitting.

Hide

```
# Trying a range of x values to see how accuracy changes
for (x in 2:5) {
  top_features <- names(sort(info_train[7, -7], decreasing = TRUE))[1:x]
  tree_fit_x <- C5.0(train_set[, top_features], train_set$acceptability)
  tree_pred_x <- predict(tree_fit_x, test_set)
  acc_x <- confusionMatrix(tree_pred_x, test_set$acceptability)$overall["Accuracy"]
  cat("x =", x, "-> Accuracy:", round(acc_x, 4), "\n")
}
```

```
x = 2 -> Accuracy: 0.8  
x = 3 -> Accuracy: 0.82  
x = 4 -> Accuracy: 0.885  
x = 5 -> Accuracy: 0.93
```

Accuracy keeps going up as more features are added — x = 2 gives 80%, x = 3 gives 82%, x = 4 jumps to 88.5%, and x = 5 gets to 93%. So dropping too many features really hurts the model. Even though safety and persons had the highest MI scores alone, the other features still add useful information. This shows there's no shortcut — using more features (closer to all 6) gives better results than picking just the top few.

I'm going with the top three features - persons , lug_boot , safety - based on their mutual information scores with the target.

Decision tree with fewer features

[Hide](#)

```
tree_fit_top3 <- C5.0(train_set[, 4:6], train_set$acceptability)  
tree_pred_top3 <- predict(tree_fit_top3, test_set)  
confusionMatrix(tree_pred_top3, test_set$acceptability)
```

Confusion Matrix and Statistics

		Reference			
Prediction		unacc	acc	good	vgood
unacc		124	7	0	0
acc		15	39	8	7
good		0	0	0	0
vgood		0	0	0	0

Overall Statistics

Accuracy : 0.815
95% CI : (0.7541, 0.8663)
No Information Rate : 0.695
P-Value [Acc > NIR] : 8.287e-05

Kappa : 0.6025

Mcnemar's Test P-Value : NA

Statistics by Class:

	Class: unacc	Class: acc	Class: good	Class: vgood
Sensitivity	0.8921	0.8478	0.00	0.000
Specificity	0.8852	0.8052	1.00	1.000
Pos Pred Value	0.9466	0.5652	NaN	NaN
Neg Pred Value	0.7826	0.9466	0.96	0.965
Prevalence	0.6950	0.2300	0.04	0.035
Detection Rate	0.6200	0.1950	0.00	0.000
Detection Prevalence	0.6550	0.3450	0.00	0.000
Balanced Accuracy	0.8887	0.8265	0.50	0.500

Using only the top 3 features (persons, lug_boot, safety) drops accuracy quite a bit — down to 81.5% and Kappa 0.60, compared to 96.5% with all 6 features. good and vgood are both completely missed now, sensitivity 0.00 for both. So even though persons and safety had the highest MI scores on their own, dropping buying and maint clearly hurts, since price and maintenance cost still add useful information the model needs, especially for spotting the smaller classes.

Naive Bayes with fewer features

[Hide](#)

```
bayes_fit_top3 <- naiveBayes(train_set[, 4:6], train_set$acceptability)
bayes_pred_top3 <- predict(bayes_fit_top3, test_set)
confusionMatrix(bayes_pred_top3, test_set$acceptability)
```

Confusion Matrix and Statistics

Reference				
Prediction	unacc	acc	good	vgood
unacc	128	17	3	0
acc	11	29	5	7
good	0	0	0	0
vgood	0	0	0	0

Overall Statistics

Accuracy : 0.785
95% CI : (0.7215, 0.8398)
No Information Rate : 0.695
P-Value [Acc > NIR] : 0.00286

Kappa : 0.4952

McNemar's Test P-Value : NA

Statistics by Class:

	Class: unacc	Class: acc	Class: good	Class: vgood
Sensitivity	0.9209	0.6304	0.00	0.000
Specificity	0.6721	0.8506	1.00	1.000
Pos Pred Value	0.8649	0.5577	NaN	NaN
Neg Pred Value	0.7885	0.8851	0.96	0.965
Prevalence	0.6950	0.2300	0.04	0.035
Detection Rate	0.6400	0.1450	0.00	0.000
Detection Prevalence	0.7400	0.2600	0.00	0.000
Balanced Accuracy	0.7965	0.7405	0.50	0.500

Naive Bayes drops even more with fewer features — 78.5% accuracy, Kappa only 0.50. Both good and vgood are missed completely again, sensitivity 0.00. So the feature drop hurts Naive Bayes worse than it hurt the decision tree, since Naive Bayes already had a weaker starting point with vgood.

Logistic regression with fewer features

[Hide](#)

```
logit_fit_top3 <- multinom(acceptability ~ persons + lug_boot + safety, data = train_set)
```

```
# weights: 32 (21 variable)
initial value 1109.035489
iter 10 value 405.899932
iter 20 value 366.665600
iter 30 value 365.504730
iter 40 value 365.448143
iter 50 value 365.423847
iter 60 value 365.410455
iter 70 value 365.401601
final value 365.401411
converged
```

Hide

```
logit_pred_top3 <- predict(logit_fit_top3, test_set)
confusionMatrix(logit_pred_top3, test_set$acceptability)
```

Confusion Matrix and Statistics

	Reference			
Prediction	unacc	acc	good	vgood
unacc	124	7	0	0
acc	15	39	8	7
good	0	0	0	0
vgood	0	0	0	0

Overall Statistics

Accuracy : 0.815
 95% CI : (0.7541, 0.8663)
 No Information Rate : 0.695
 P-Value [Acc > NIR] : 8.287e-05

Kappa : 0.6025

Mcnemar's Test P-Value : NA

Statistics by Class:

	Class: unacc	Class: acc	Class: good	Class: vgood
Sensitivity	0.8921	0.8478	0.00	0.000
Specificity	0.8852	0.8052	1.00	1.000
Pos Pred Value	0.9466	0.5652	NaN	NaN
Neg Pred Value	0.7826	0.9466	0.96	0.965
Prevalence	0.6950	0.2300	0.04	0.035
Detection Rate	0.6200	0.1950	0.00	0.000
Detection Prevalence	0.6550	0.3450	0.00	0.000
Balanced Accuracy	0.8887	0.8265	0.50	0.500

Logistic regression with fewer features gets 81.5% accuracy, Kappa 0.60 — same numbers as the decision tree with fewer features, and same pattern too: good and vgood both completely missed, sensitivity 0.00. So again, dropping buying and maint costs the model its ability to spot the smaller classes, even though the remaining features had the highest MI scores individually.

SVM with fewer features

Hide

```
svm_fit_top3 <- ksvm(acceptability ~ persons + lug_boot + safety, data = train_set, kernel = "v
anilladot")
```

Setting default kernel parameters

Hide

```
svm_pred_top3 <- predict(svm_fit_top3, test_set)
confusionMatrix(svm_pred_top3, test_set$acceptability)
```

Confusion Matrix and Statistics

Reference				
Prediction	unacc	acc	good	vgood
unacc	128	16	2	0
acc	11	30	6	7
good	0	0	0	0
vgood	0	0	0	0

Overall Statistics

Accuracy : 0.79
95% CI : (0.7269, 0.8443)
No Information Rate : 0.695
P-Value [Acc > NIR] : 0.001704

Kappa : 0.5123

Mcnemar's Test P-Value : NA

Statistics by Class:

	Class: unacc	Class: acc	Class: good	Class: vgood
Sensitivity	0.9209	0.6522	0.00	0.000
Specificity	0.7049	0.8442	1.00	1.000
Pos Pred Value	0.8767	0.5556	NaN	NaN
Neg Pred Value	0.7963	0.8904	0.96	0.965
Prevalence	0.6950	0.2300	0.04	0.035
Detection Rate	0.6400	0.1500	0.00	0.000
Detection Prevalence	0.7300	0.2700	0.00	0.000
Balanced Accuracy	0.8129	0.7482	0.50	0.500

SVM with fewer features drops to 79% accuracy, Kappa 0.51 — the biggest drop of all the models tried with just 3 features so far. good and vgood are both completely missed again, sensitivity 0.00. So SVM struggles the most when features are reduced, even worse than Naive Bayes and the decision tree did.

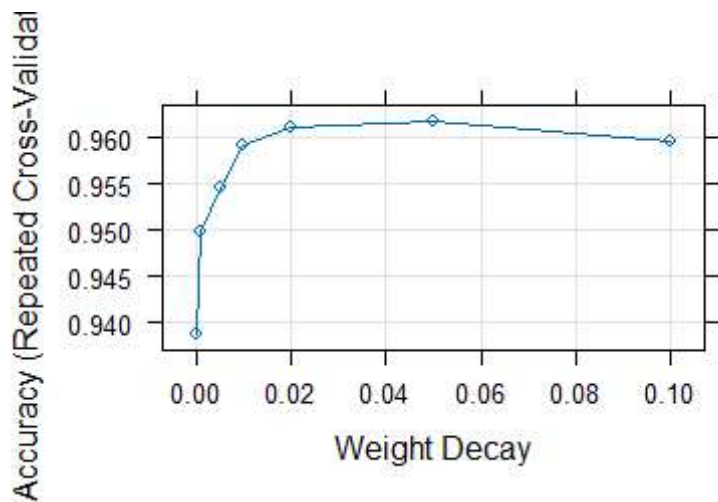
Step 2b: Squeezing out more performance

Tuning the neural network's decay rate

[Hide](#)

```
tune_ctrl <- trainControl(method = "repeatedcv", number = 10, repeats = 10)
net_grid <- expand.grid(size = 5, decay = c(0, 0.001, 0.005, 0.01, 0.02, 0.05, 0.1))
set.seed(123)

net_tuned <- train(train_set[, -7], train_set[, 7], method = "nnet",
                  trControl = tune_ctrl, tuneGrid = net_grid, trace = FALSE)
plot(net_tuned)
```



Accuracy is lowest at decay = 0 (below 0.94), jumps up fast, then peaks around decay = 0.02–0.05 (close to 0.962), and starts dropping again by decay = 0.1. So a small amount of decay helps the network avoid overfitting, but too much decay hurts it again. The sweet spot looks to be somewhere around 0.02–0.05.

Tuning the decision tree's number of trials

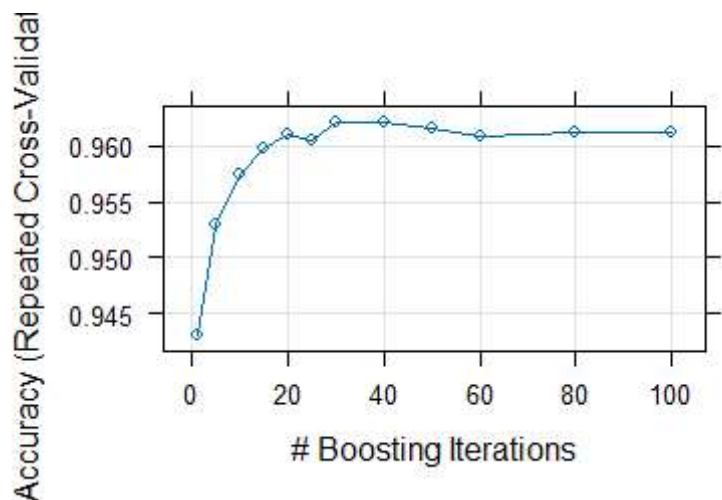
[Hide](#)

```
tune_ctrl <- trainControl(method = "repeatedcv", number = 10, repeats = 10)
tree_grid <- expand.grid(model = "tree",
                        trials = c(1, 5, 10, 15, 20, 25, 30, 40, 50, 60, 80, 100),
                        winnow = FALSE)

set.seed(123)

tree_tuned <- train(train_set[, -7], train_set[, 7], method = "C5.0",
                  trControl = tune_ctrl, tuneGrid = tree_grid)

plot(tree_tuned)
```



Accuracy starts low with trials = 1 (below 0.945), climbs fast up to around trials = 20-30 (about 0.962), then flattens out and stays roughly the same all the way to trials = 100. So more boosting rounds help at first, but after about 20-30 trials, adding more doesn't really improve anything — the model has already learned what it can from this data.

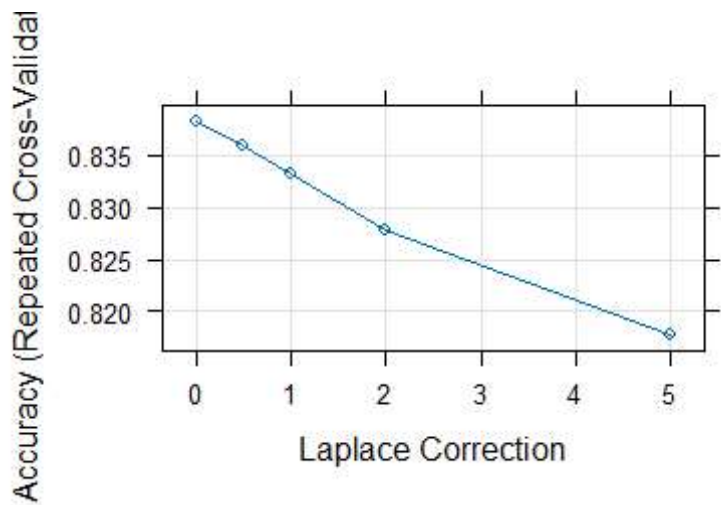
Tuning naive Bayes's fL (Laplace correction)

Hide

```
tune_ctrl <- trainControl(method = "repeatedcv", number = 10, repeats = 10)
bayes_grid <- expand.grid(fL = c(0, 0.5, 1, 2, 5), usekernel = FALSE, adjust = 1)
set.seed(123)

bayes_tuned <- train(train_set[, -7], train_set[, 7], method = "nb",
                  trControl = tune_ctrl, tuneGrid = bayes_grid)

plot(bayes_tuned)
```



Accuracy is highest at $fL = 0$ (close to 0.84), and just keeps dropping as fL goes up, down to about 0.82 at $fL = 5$. So unlike the neural network or decision tree, adding Laplace correction here doesn't help at all — it actually makes things worse. The default $fL = 0$ turns out to be the best choice for this dataset.

Part 3. Presentation of Specified Results

(To be finalised in the lab once the exact figures required are confirmed.)

References

- Kuhn, M. (2008) 'Building predictive models in R using the caret package', *Journal of Statistical Software*, 28(5), pp. 1-26.
- UCI Machine Learning Repository (1997) *Car Evaluation Data Set*. Available at: <https://archive.ics.uci.edu/dataset/19/car+evaluation> (<https://archive.ics.uci.edu/dataset/19/car+evaluation>) (Accessed: [17 August 2026]).
- Wickham, H. (2016) *ggplot2: Elegant Graphics for Data Analysis*. New York: Springer-Verlag.